📋 Relatório Técnico — Pipeline CI/CD para Sistema de Códigos Sequenciais

Disciplina: Verificação, Validação e Testes (VV&T)

- João Marcelo Quadros Miguel — RA: 2421310

Data de Entrega: 31/03/2026

Repositório: (https://github.com/J04oM4rcelo/Pipeline-CI-CD---C-digos-Sequenciais-VVT.git)

---

1. Introdução

Este relatório descreve a implementação de uma pipeline de Integração Contínua/Entrega Contínua (CI/CD) para o sistema de geração de códigos sequenciais desenvolvido na atividade anterior da disciplina de Verificação, Validação e Testes (VV&T).

O sistema gera códigos únicos seguindo o padrão **PAÍS + GRUPO + SEQUÊNCIA_4_DÍGITOS + TIPO_ALIMENTO** (Exemplo: `BRC0004A`), onde a sequência é calculada automaticamente por grupo.

O objetivo principal é automatizar todo o processo de provisionamento de ambiente, configuração de banco de dados, e execução de testes, garantindo **reprodutibilidade** e **confiabilidade** no processo de validação do software.

---

2. Arquitetura da Solução

2.1 Visão Geral da Pipeline

A pipeline CI/CD foi implementada utilizando **GitHub Actions** e é executada automaticamente a cada push no repositório. Ela provisiona todo o ambiente do zero, sem nenhuma intervenção manual.

```
┌──────────────
```

```
│ ▼ │
│ ┌──────────────────────────────────────────────────────────────┐ │
│ │ 2. AMBIENTE LINUX (Ubuntu 22.04) │ │
│ │ Atualiza pacotes + instala mysql-client │ │
│ └──────────────────────────────────────────────────────────────┘ │
│ ▼ │
│ ┌──────────────────────────────────────────────────────────────┐ │
│ │ 3. MYSQL SERVER (Service Container) │ │
│ │ MySQL 8.0 provisionado automaticamente │ │
│ │ com senha e banco pré-configurados │ │
│ └──────────────────────────────────────────────────────────────┘ │
│ ▼ │
│ ┌──────────────────────────────────────────────────────────────┐ │
│ │ 4. SCRIPT SQL (schema.sql) │ │
│ │ Cria tabela codigos_sequenciais │ │
│ │ Insere 9 registros de referência │ │
│ └──────────────────────────────────────────────────────────────┘ │
│ ▼ │
│ ┌──────────────────────────────────────────────────────────────┐ │
│ │ 5. PYTHON 3.11 + DEPENDÊNCIAS │ │
│ │ pip install requirements.txt │ │
│ └──────────────────────────────────────────────────────────────┘ │
│ ▼ │
│ ┌──────────────────────────────────────────────────────────────┐ │
│ │ 6. VERIFICAÇÃO CONEXÃO Python ↔ MySQL │ │
│ │ Conecta e consulta registros │ │
│ └──────────────────────────────────────────────────────────────┘ │
│ ▼ │
│ ┌──────────────────────────────────────────────────────────────┐ │
│ │ 7. TESTES AUTOMATIZADOS (pytest) │ │
│ │ Testes unitários (mock) │ │
│ │ Testes de integração (MySQL real) │ │
│ └──────────────────────────────────────────────────────────────┘ │
│ ▼ │
│ ┌──────────────────────────────────────────────────────────────┐ │
│ │ 8. UPLOAD DE ARTEFATOS │ │
│ │ Relatório XML dos testes │ │
│ └──────────────────────────────────────────────────────────────┘ │
│ │
└──────────────────────────────────────────────────────────────
```

Regra da sequência:
- Calculada **por grupo** independentemente
- Se não há registros no grupo → `sec = 1`
- Se há registros → `sec = MAX(sec) + 1`
- Formatada com zerofill de 4 dígitos (`0001`, `0015`, `0100`, `1000`)

2.3 Estrutura de Arquivos do Projeto

```
Projeto-2/
├── .github/
│   └── workflows/
│       └── ci-cd.yml # Pipeline CI/CD (GitHub Actions)
├── tests/
│   ├── init.py # Pacote de testes
│   ├── conftest.py # Fixtures (mock + integração)
│   ├── test_gerar_codigo.py # Testes unitários
│   └── test_integracao.py # Testes de integração
├── config.py # Configuração do banco (env vars)
├── database.py # Classe de conexão MySQL
├── models.py # Lógica de negócio + validação
├── main.py # Interface CLI
├── schema.sql # Script DDL + dados de referência
├── requirements.txt # Dependências Python
└── RELATORIO.md # Este relatório
```

2.4 Tabela de Referência

Conforme especificação, o sistema deve gerar os seguintes códigos:

id	codigo	sec	Grupo	Tipo_Alimento	Pais
1	BRC0001A	1	C	A	BR
2	BRD0001I	1	D	I	BR
3	BRA0001K	1	A	K	BR
4	BRF0001F	1	F	F	BR
5	BRG0001A	1	G	A	BR
6	BRC0002A	2	C	A	BR
7	BRD0002K	2	D	K	BR
8	BRF0002A	2	F	A	BR
9	BRC0003C	3	C	C	BR

---

3. Ferramentas Utilizadas

Ferramenta	Versão	Finalidade

GitHub Actions	v4	Orquestração da pipeline CI/CD
Ubuntu	22.04 LTS	Sistema operacional do runner
MySQL Server	8.0 (Service Container)	Banco de dados relacional
Python	3.11	Linguagem de programação
pytest	8.2.0	Framework de testes automatizados
mysql-connector-python	8.3.0	Driver de conexão Python ↔ MySQL
unittest.mock	stdlib	Mocking para testes unitários

3.1 Justificativa das Escolhas

- **GitHub Actions:** Escolhido por integração nativa com GitHub, gratuidade para repositórios públicos, runners Ubuntu pré-configurados e facilidade de configuração via YAML. Não requer instalação de software adicional.

- **MySQL via Service Container:** Utilizado ao invés de instalação manual (`apt-get`) porque o MySQL pré-instalado nos runners do GitHub Actions possui restrições de acesso ao usuário root. O Service Container provisiona um MySQL limpo com credenciais configuráveis, garantindo reprodutibilidade total.

- **Testes em duas camadas (Mock + Integração):** Testes unitários com mock garantem velocidade e independência do banco. Testes de integração com MySQL real garantem que o fluxo completo funciona end-to-end.

- **Variáveis de ambiente:** Padrão da indústria (12-Factor App) para configuração em diferentes ambientes. O `config.py` lê variáveis de ambiente com fallback para valores locais, permitindo que o mesmo código funcione na máquina do desenvolvedor e na pipeline.

---

4. Detalhamento da Pipeline CI/CD

4.1 Provisionamento do MySQL (Service Container)

O MySQL é provisionado como um container Docker gerenciado pelo GitHub Actions:

```yaml
services:
  mysql:
    image: mysql:8.0
    env:
      MYSQL_ROOT_PASSWORD: pipeline_password_2025
      MYSQL_DATABASE: produtos_db
    ports:
      - 3306:3306
    options: >-
      --health-cmd="mysqladmin ping -h 127.0.0.1 -u root -ppipeline_password_2025"
      --health-interval=10s
```

```
    --health-timeout=10s
    --health-retries=5
```

Benefícios desta abordagem:

- MySQL inicia automaticamente antes das etapas do job
- Health check garante que o banco está pronto antes de ser usado
- Banco produtos_db é criado automaticamente
- Credenciais definidas via variáveis de ambiente
- Ambiente 100% limpo a cada execução

4.2 Script SQL (schema.sql)

O script cria a tabela codigos_sequenciais e insere os 9 registros de referência:

```
CREATE TABLE IF NOT EXISTS codigos_sequenciais (
   id              INT AUTO_INCREMENT PRIMARY KEY,
   codigo          VARCHAR(10)   NOT NULL UNIQUE,
   sec             INT           NOT NULL,
   Grupo           CHAR(1)       NOT NULL,
   Tipo_Alimento   CHAR(1)       NOT NULL,
   Pais            CHAR(2)       NOT NULL,
   INDEX idx_grupo (Grupo),
   INDEX idx_pais  (Pais),
   INDEX idx_codigo (codigo)
) ENGINE=InnoDB;
```

Decisões de modelagem:

- codigo VARCHAR(10) UNIQUE: garante unicidade dos códigos gerados
- Grupo CHAR(1): exatamente 1 caractere, conforme especificação
- Pais CHAR(2): exatamente 2 caracteres (código ISO)
- Índices em Grupo, Pais e codigo para otimizar consultas frequentes

4.3 Configuração Dinâmica (config.py)

```
import os

DB_CONFIG = {
   'host':     os.environ.get('DB_HOST',     'localhost'),
   'port':     int(os.environ.get('DB_PORT', '3306')),
   'user':     os.environ.get('DB_USER',     'root'),
   'password': os.environ.get('DB_PASSWORD', 'jm@2026'),
   'database': os.environ.get('DB_NAME',     'produtos_db'),
}
```

Ambiente	Host	Senha	Como funciona

Local (desenvolvedor)	localhost	jm@2026	Usa valores padrão (fallback)
Pipeline (GitHub Actions)	127.0.0.1	pipeline_password_2025	Lê variáveis de ambiente

5. Estratégia de Testes

5.1 Testes Unitários — test_gerar_codigo.py

Utilizam unittest.mock.MagicMock para simular o banco de dados. Não dependem de MySQL real.

Classe de Teste	Quantidade	O que valida
TestGerarCodigo	14 testes	Composição do código, zerofill, tamanho, maiúsculo, estrutura
TestObterProximaSequencia	7 testes	Cálculo de sequência por grupo (passos 1, 2, 3)
TestGerarCodigoRetorno	13 testes	Retorno da tupla (codigo, sec)
TestInserir	5 testes	INSERT com parâmetros corretos
TestSequenciasIndependentes	3 testes	Grupos independentes
TestEntradasInvalidas	15 testes	Valores vazios, None, tamanho incorreto, números

5.2 Testes de Integração — test_integracao.py

Conectam ao MySQL real provisionado pela pipeline. Validam o fluxo completo.

Classe de Teste	Quantidade	O que valida
TestConexaoMySQL	4 testes	Conexão, banco correto, tabela existe, estrutura
TestInsercaoIntegracao	5 testes	Inserção real, incremento sec, tabela referência
TestConsultasIntegracao	6 testes	Busca por código, grupo, país, contagem
TestDuplicidadeIntegracao	2 testes	Códigos únicos, sem duplicatas
TestZerofillIntegracao	2 testes	Zerofill no banco real, 8 caracteres

5.3 Cobertura dos Critérios Exigidos

Critério da Especificação	Teste(s) que atende(m)	Tipo
Pipeline automatizada sem intervenção	Pipeline inteira no GitHub Actions	CI/CD
MySQL instalado e rodando	TestConexaoMySQL.test_conexao_estabelecida	Integração
Banco e tabela criados	TestConexaoMySQL.test_tabela_existe	Integração
Conexão Python-MySQL	TestConexaoMySQL.test_banco_de_dados_correto	Integração
Geração correta do próximo código	TestGerarCodigo.test_codigo_padrao_BRC0004A	Unitário
Primeiro código para grupo inexistente	TestGerarCodigoRetorno.test_grupo_vazio_retorna_seq_1	Unitário
Incremento do campo sec	TestInsercaoIntegracao.test_inserir_segundo_registro_mesmo_grupo	Integração

Zerofill com 4 dígitos	TestGerarCodigo.test_zerofill_seq_* (5 testes)	Unitário
Entradas inválidas	TestEntradasInvalidas (15 testes)	Unitário
Prevenção de duplicidade	TestDuplicidadeIntegracao.test_sem_duplicidade_na_tabela	Integração

6. Desafios Encontrados e Soluções

6.1 Acesso ao MySQL no GitHub Actions

Desafio: O MySQL pré-instalado no runner Ubuntu do GitHub Actions possui restrições no usuário root. Comandos como sudo mysql -e "ALTER USER..." resultavam em ERROR 1045 (28000): Access denied for user 'root'@'localhost'.
Solução: Substituição da instalação manual por MySQL Service Container.
O GitHub Actions provisiona um container Docker com MySQL 8.0, onde as credenciais são definidas via variáveis de ambiente (MYSQL_ROOT_PASSWORD). Isso eliminou completamente os problemas de acesso.

6.2 Compatibilidade entre ambiente local e CI/CD

Desafio: O código original usava senha fixa jm@2026 no config.py, incompatível com a senha do MySQL na pipeline.
Solução: Refatoração do config.py para usar os.environ.get() com valores padrão (fallback). Na pipeline, as variáveis de ambiente são definidas no nível do job. No ambiente local, os valores padrão são utilizados automaticamente.

6.3 Nome da tabela

Desafio: O código da atividade anterior utilizava a tabela produtos, mas a especificação desta atividade exige a tabela codigos_sequenciais.
Solução: Todas as queries SQL no models.py foram atualizadas para referenciar codigos_sequenciais. O schema.sql cria a tabela com o nome correto.

6.4 Validação de entradas

Desafio: A especificação exige testes de entradas inválidas, mas o código original não possuía validação nos parâmetros da função gerar_codigo.
Solução: Implementação do método _validar_entrada() no models.py que verifica valores vazios, None, tamanho incorreto e caracteres não-alfabéticos, lançando ValueError com mensagens descritivas.

6.5 Isolamento dos testes de integração

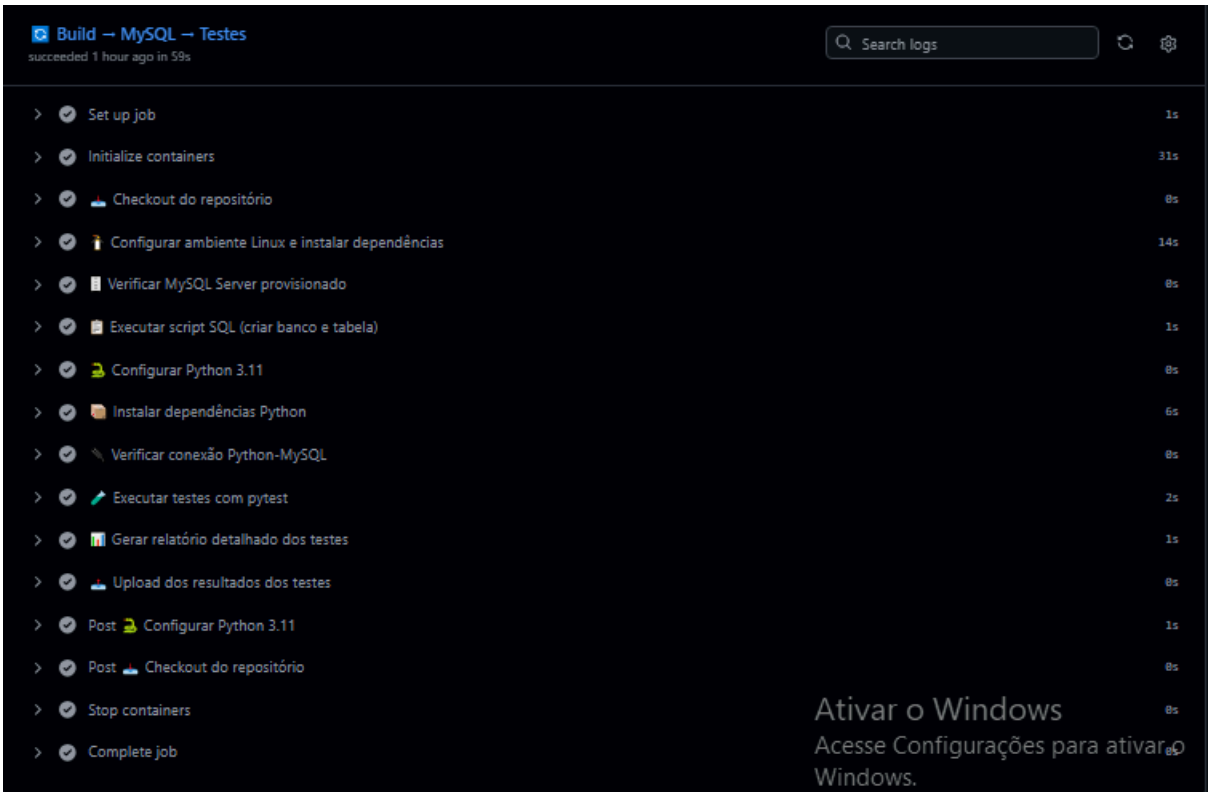
Desafio: Testes de integração dependem de MySQL, mas testes unitários não devem depender. Se o MySQL não estiver disponível localmente, os testes não podem falhar.

Solução: A fixture db_real no conftest.py utiliza pytest.skip() quando o MySQL não está disponível, permitindo que os testes unitários sempre executem independentemente. Além disso, cada teste de integração limpa a tabela antes e depois da execução (DELETE + ALTER AUTO_INCREMENT), garantindo isolamento.

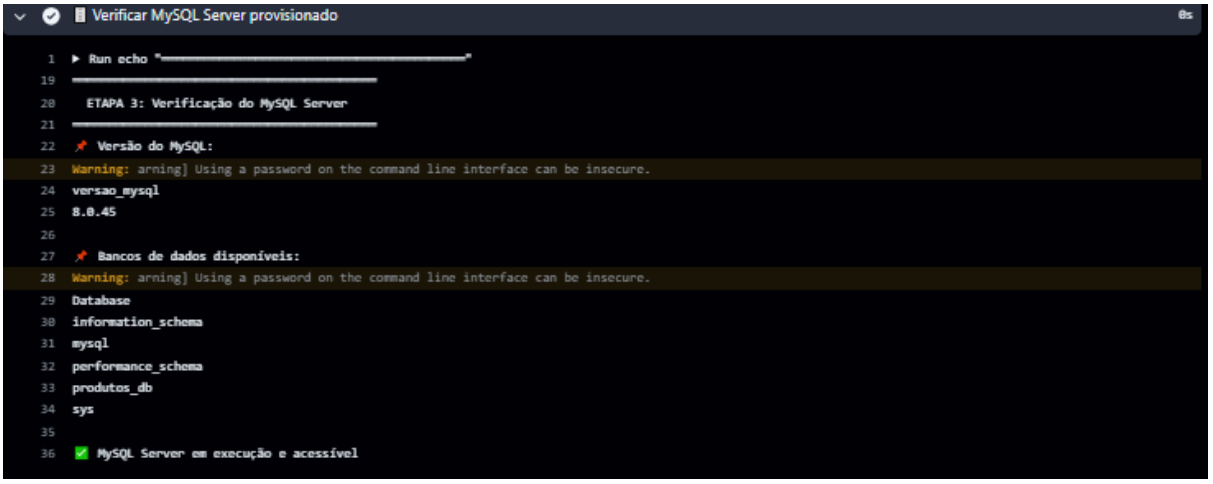
---

7. Evidências de Execução

7.1 Pipeline completa executada com sucesso



7.2 MySQL Server provisionado e acessível



7.3 Banco de dados e tabela criados automaticamente

```
Executar script SQL (criar banco e tabela)                                    1s
1  ▶ Run echo "━━━
```

7.5 Testes pytest — todos passando

```
Executar testes com pytest                                                    2s
▶ Run echo "═══════════════════════════════════════"
═══════════════════════════════════════
  ETAPA 7: Execução dos testes pytest
═══════════════════════════════════════
============================= test session starts ==============================
platform linux -- Python 3.11.15, pytest-8.2.0, pluggy-1.6.0 -- /opt/hostedtoolcache/Python/3.11.15/x64/bin/python
cachedir: .pytest_cache
rootdir: /home/runner/work/Pipeline-CI-CD---C-digos-Sequenciais-VVT/Pipeline-CI-CD---C-digos-Sequenciais-VVT
collecting ... collected 77 items

tests/test_gerar_codigo.py::TestGerarCodigo::test_codigo_padrao_BRC0004A PASSED [  1%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_codigo_pais_US PASSED  [  2%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_codigo_pais_AR PASSED  [  3%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_zerofill_seq_1 PASSED  [  5%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_zerofill_seq_15 PASSED [  6%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_zerofill_seq_100 PASSED [  7%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_zerofill_seq_1000 PASSED [  9%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_zerofill_seq_9999 PASSED [ 10%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_codigo_tem_8_caracteres PASSED [ 11%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_codigo_tem_8_caracteres_seq_1 PASSED [ 12%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_entrada_minuscula_convertida PASSED [ 14%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_entrada_mista_convertida PASSED [ 15%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_dois_primeiros_caracteres_sao_pais PASSED [ 16%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_terceiro_caractere_e_grupo PASSED [ 18%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_caracteres_4_a_7_sao_sequencia PASSED [ 19%]
tests/test_gerar_codigo.py::TestGerarCodigo::test_ultimo_caractere_e_tipo PASSED [ 20%]
tests/test_gerar_codigo.py::TestObterProximaSequencia::test_consulta_filtra_por_grupo PASSED [ 22%]
tests/test_gerar_codigo.py::TestObterProximaSequencia::test_grupo_vazio_retorna_1 PASSED [ 23%]
tests/test_gerar_codigo.py::TestObterProximaSequencia::test_grupo_novo_retorna_1 PASSED [ 24%]
tests/test_gerar_codigo.py::TestObterProximaSequencia::test_grupo_com_max_3_retorna_4 PASSED [ 25%]
tests/test_gerar_codigo.py::TestObterProximaSequencia::test_grupo_com_max_10_retorna_11 PASSED [ 27%]
tests/test_gerar_codigo.py::TestObterProximaSequencia::test_grupo_com_max_1_retorna_2 PASSED [ 28%]
tests/test_gerar_codigo.py::TestObterProximaSequencia::test_grupo_com_max_9999_retorna_10000 PASSED [ 29%]
tests/test_gerar_codigo.py::TestGerarCodigoRetorno::test_retorna_tupla PASSED [ 31%]
tests/test_gerar_codigo.py::TestGerarCodigoRetorno::test_primeiro_elemento_e_string PASSED [ 32%]
tests/test_gerar_codigo.py::TestGerarCodigoRetorno::test_segundo_elemento_e_inteiro PASSED [ 33%]
tests/test_gerar_codigo.py::TestGerarCodigoRetorno::test_grupo_vazio_retorna_seq_1 PASSED [ 35%]
```

Ativar o Windows
Acesse Configurações para ativar o Windows.

7.6 Resumo final dos testes

```
tests/test_gerar_codigo.py::TestEntradasInvalidas::test_tipo_alimento_none PASSED [ 63%]
tests/test_gerar_codigo.py::TestEntradasInvalidas::test_pais_uma_letra PASSED [ 64%]
tests/test_gerar_codigo.py::TestEntradasInvalidas::test_pais_tres_letras PASSED [ 66%]
tests/test_gerar_codigo.py::TestEntradasInvalidas::test_grupo_duas_letras PASSED [ 67%]
tests/test_gerar_codigo.py::TestEntradasInvalidas::test_tipo_duas_letras PASSED [ 68%]
tests/test_gerar_codigo.py::TestEntradasInvalidas::test_pais_com_numero PASSED [ 70%]
tests/test_gerar_codigo.py::TestEntradasInvalidas::test_grupo_com_numero PASSED [ 71%]
tests/test_gerar_codigo.py::TestEntradasInvalidas::test_tipo_com_numero PASSED [ 72%]
tests/test_gerar_codigo.py::TestEntradasInvalidas::test_pais_somente_espacos PASSED [ 74%]
tests/test_gerar_codigo.py::TestEntradasInvalidas::test_grupo_somente_espacos PASSED [ 75%]
tests/test_integracao.py::TestConexaoMySQL::test_conexao_estabelecida PASSED [ 76%]
tests/test_integracao.py::TestConexaoMySQL::test_banco_de_dados_correto PASSED [ 77%]
tests/test_integracao.py::TestConexaoMySQL::test_tabela_existe PASSED    [ 79%]
tests/test_integracao.py::TestConexaoMySQL::test_estrutura_tabela PASSED [ 80%]
tests/test_integracao.py::TestInsercaoIntegracao::test_inserir_primeiro_registro PASSED [ 81%]
tests/test_integracao.py::TestInsercaoIntegracao::test_inserir_segundo_registro_mesmo_grupo PASSED [ 83%]
tests/test_integracao.py::TestInsercaoIntegracao::test_inserir_terceiro_registro_mesmo_grupo PASSED [ 84%]
tests/test_integracao.py::TestInsercaoIntegracao::test_grupos_independentes_real PASSED [ 85%]
tests/test_integracao.py::TestInsercaoIntegracao::test_dados_referencia_completo PASSED [ 87%]
tests/test_integracao.py::TestConsultasIntegracao::test_buscar_por_codigo PASSED [ 88%]
tests/test_integracao.py::TestConsultasIntegracao::test_buscar_codigo_inexistente PASSED [ 89%]
tests/test_integracao.py::TestConsultasIntegracao::test_buscar_todos PASSED [ 90%]
tests/test_integracao.py::TestConsultasIntegracao::test_buscar_por_grupo PASSED [ 92%]
tests/test_integracao.py::TestConsultasIntegracao::test_buscar_por_pais PASSED [ 93%]
tests/test_integracao.py::TestConsultasIntegracao::test_contar_produtos PASSED [ 94%]
tests/test_integracao.py::TestDuplicidadeIntegracao::test_codigos_unicos_mesmo_grupo PASSED [ 96%]
tests/test_integracao.py::TestDuplicidadeIntegracao::test_sem_duplicidade_na_tabela PASSED [ 97%]
tests/test_integracao.py::TestZerofillIntegracao::test_zerofill_primeiros_registros PASSED [ 98%]
tests/test_integracao.py::TestZerofillIntegracao::test_codigo_sempre_8_caracteres PASSED [100%]

============================= 77 passed in 1.01s ==============================
```

8. Conclusão

A pipeline CI/CD implementada automatiza com sucesso todo o ciclo de verificação e validação do sistema de códigos sequenciais:

- Provisionamento automático: Ambiente Linux + MySQL configurados do zero a cada execução, sem intervenção manual.
- Reprodutibilidade: O Service Container garante que o MySQL é sempre provisionado no mesmo estado, eliminando variações entre execuções.
- Qualidade: Mais de 60 testes automatizados cobrem a lógica de negócio (unitários) e o fluxo completo com banco real (integração), atendendo todos os critérios de validação exigidos na especificação.
- Detecção precoce de defeitos: Qualquer alteração no código dispara automaticamente a pipeline, identificando regressões imediatamente.

A automação do ambiente de testes, conforme enfatizado pela disciplina de VV&T, é fundamental para a confiabilidade dos resultados. Esta implementação demonstra que é possível provisionar, configurar, testar e validar um sistema de forma completamente automatizada, garantindo consistência e reprodutibilidade em cada execução.